Práctica de aprendizaje automático (parte 3)

Autores: Guillem Borràs y Gonzalo Moll

Entrega este cuaderno habiendo **ejecutado todas las celdas**. Incluye en la entrega **todos los ficheros necesarios para su ejecución**.

Asegúrate de que la presentación está bien estructurada: Se valorará la **claridad, concisión, y completitud** del informe.

Se trata de una práctica abierta: **sé intrépido y explora**.

1. Describe las características de los datos:
    - Tipo de datos (e.g. https://archive.ics.uci.edu/ml/datasets/statlog+(german+credit+data)))
    - Número de ejemplos de cada una de las clases.
    - Número y tipo (nominales no ordenados, nominales ordenados, numéricos) de atributos.

Esta información se debería utilizar en el análisis; por ejemplo, si se selecciona un atributo para la predicción, ¿es clara su relevancia por el tipo de información que proporciona de acuerdo con nuestro conocimiento experto sobre el área?

2. Detalla la metodología utilizada:
    - Partitición de los datos: tamaño de los conjuntos de entrenamiento y test, uso de estratificación en el muestreo.
    - Preprocesamiento: condificación de los atributos, construcción y selección de características, normalización, etc. (¡solo se debe utilizar la información del consjunto de entrenamiento!)
    - Determinación de los hiperparámetros; por ejemplo, mediante búsqueda en rejilla y validación cruzada.
    - Estimación del error de generalización y su incertidumbre.
3. Resume los resultados en gráficas y tablas.
4. Elabora esta información e ilustra tus observaciones con los resultados obtenidos.
5. Proprociona una recomendación final:
    - Tipo de clasificador.
    - Configuración del clasificador (arquitectura, hiperparámetros, etc.) y método de entrenamiento (función de coste, método de optimización, uso de técnicas de regularización,...)
    - para el predictor final, proporciona los errores de entrenamiento, validación cruzada y test.
6. Resume las conclusiones del análisis.

In [1]:
```
%load_ext autoreload
%autoreload 2

import sys
import importlib
import warnings

# Suppress all warnings
warnings.filterwarnings('ignore')

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classifi

from utils_assignement3 import plot_confusion_matrix, VectorizerClassifierAnalyz
```

3. Procesamiento de texto y datos no balanceados (3 puntos)

In [2]:
```
df = pd.read_csv("train.tsv", sep='\t', index_col="PhraseId")
pd.set_option('display.max_colwidth', None)
df.head(2)
```

Out[2]:

	SentenceId	Phrase	Sentiment
PhraseId			
1	1	A series of escapades demonstrating the adage that what is good for the goose is also good for the gander , some of which occasionally amuses but none of which amounts to much of a story .	1
2	1	A series of escapades demonstrating the adage that what is good for the goose	2

Un ejemplo concreto de frase que se está usando en el dataset:

In [3]:
```
print(df.loc[1].Phrase)
```

```
A series of escapades demonstrating the adage that what is good for the goose is
also good for the gander , some of which occasionally amuses but none of which am
ounts to much of a story .
```

Este conjunto de datos que usaremos ahora permite aprender el sentimiento del texto a partir de frases etiquetadas. Las etiquetas para los sentimientos son:

- 0 - negative
- 1 - somewhat negative
- 2 - neutral
- 3 - somewhat positive
- 4 - positive

Más información aquí.

A lo largo de este ejercicio veremos distintas formas de procesar el texto y de equilibrar las clases a aprender.

In [4]:
```
train, test = train_test_split(df[["Phrase", "Sentiment"]], random_state=0)
X_train = train.Phrase
X_test = test.Phrase
y_train = train.Sentiment
y_test = test.Sentiment
```

Modelos básicos

Vamos a comenzar con una concatenación (*pipe*) de modelos muy básicos y analizaremos si tienen algún problema.

In [5]:
```
count_vectorizer = CountVectorizer(max_features=1000)
decision_tree = DecisionTreeClassifier(criterion='entropy', max_depth=2)
pipe = make_pipeline(count_vectorizer,decision_tree)
```

In [6]:
```
def get_accuracy(pipe):
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    return round(accuracy_score(y_pred, y_test),3)
```

In [7]:
```
get_accuracy(pipe)
```

Out[7]:
```
0.515
```

Veamos qué clases se están prediciendo de verdad

In [8]:
```
# Etiquetas de entrenamiento
round(pd.Series(y_train).value_counts(normalize=True),2)
```

Out[8]:
```
Sentiment
2    0.51
3    0.21
1    0.17
4    0.06
0    0.04
Name: proportion, dtype: float64
```

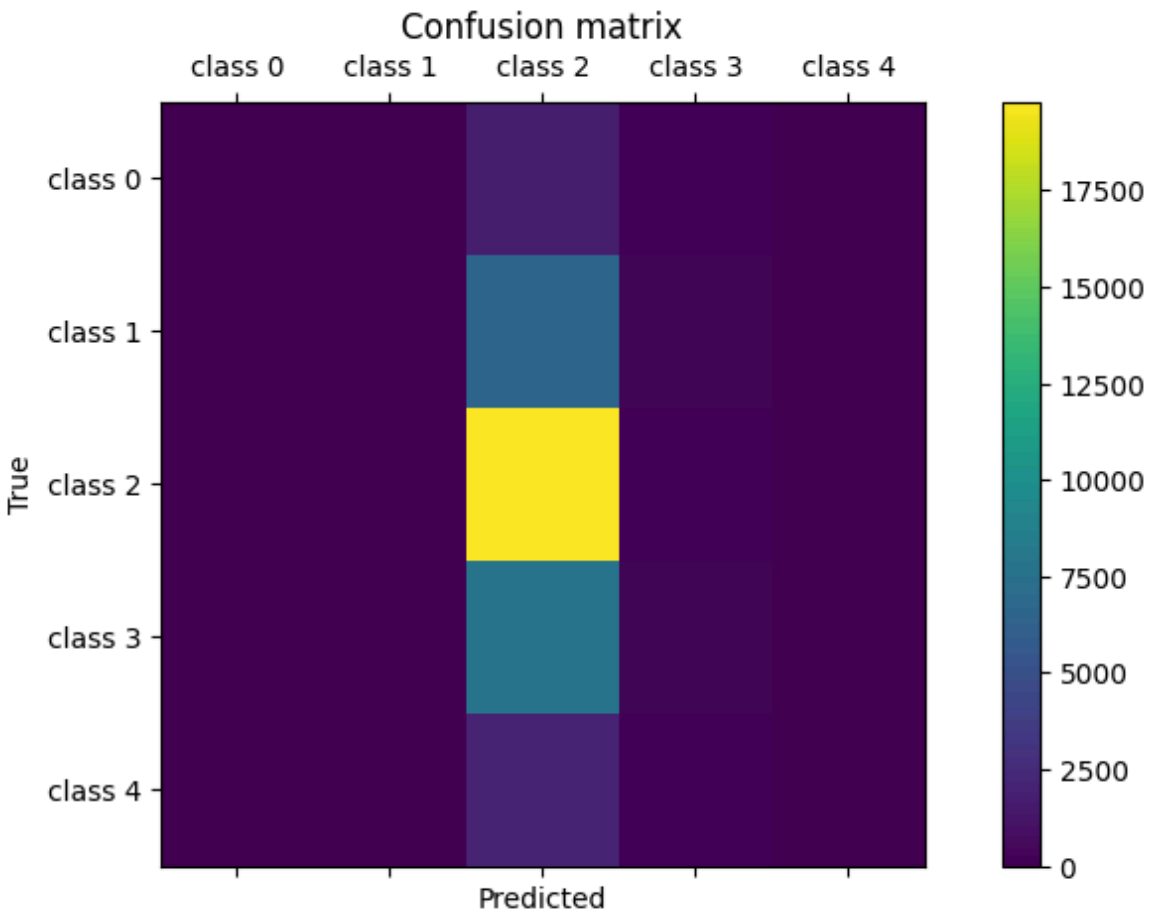
In [9]:
```
# Clases en predicción
y_pred = pipe.predict(X_test)
round(pd.Series(y_pred).value_counts(normalize=True),2)
```

Out[9]:
```
2    0.97
3    0.03
Name: proportion, dtype: float64
```

Es decir: al ser un problema no balanceado, la predicción está claramente sesgada hacia una de las clases.

Esto se puede ver también analizando la **matriz de confusión**

In [10]:
```
plot_confusion_matrix(X_test, y_test, pipe)
```



Cambio de modelos y procesamiento de los datos

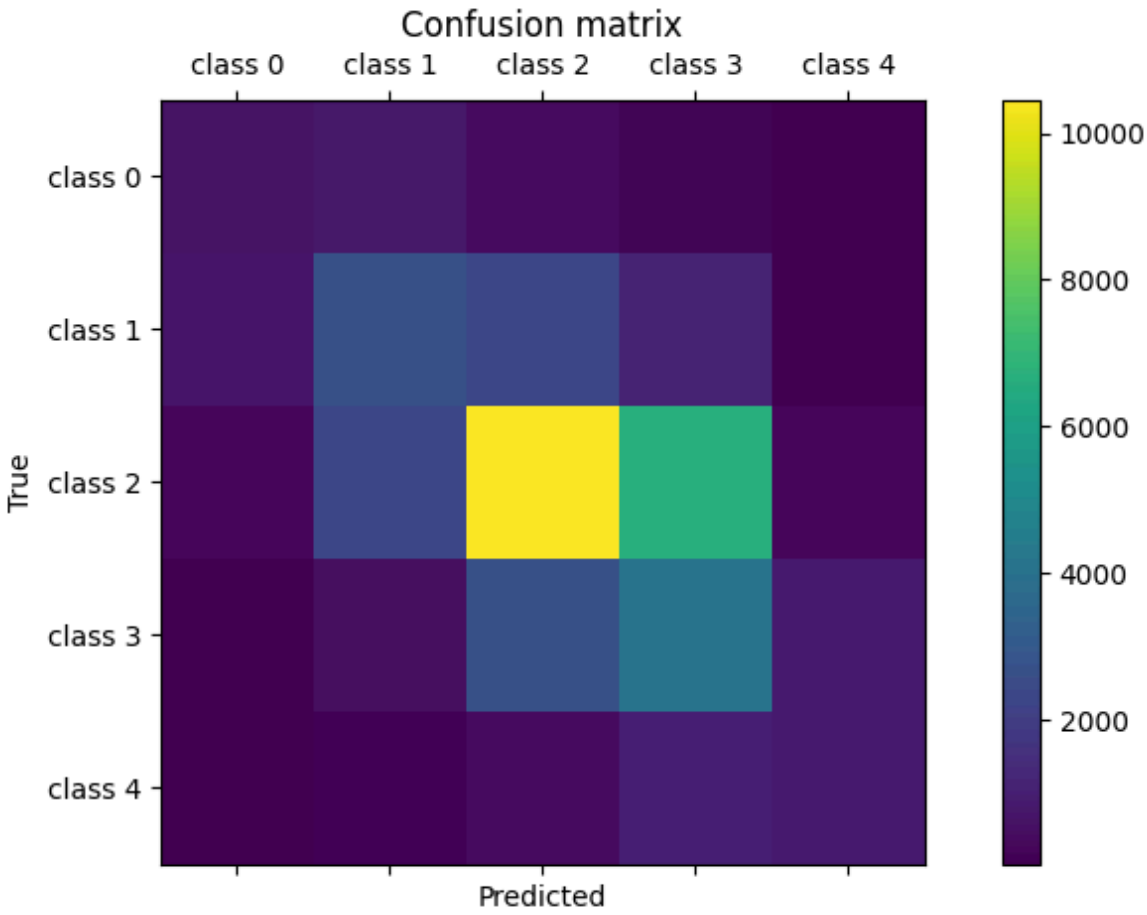
Analiza si algún otro modelo de los vistos en ejercicios anteriores evita este problema de las clases desbalanceadas.

Por ejemplo, en la celda siguiente se cambia el árbol de decisión por un KNN.

In [11]:
```
knn = KNeighborsClassifier(n_neighbors=1)
pipe_knn = make_pipeline(count_vectorizer,knn)
get_accuracy(pipe_knn)
```

Out[11]: 0.475

In [12]:
```
plot_confusion_matrix(X_test, y_test, pipe_knn)
```



Parte 1: Explora varios modelos y responde:

- ¿Con qué modelo se obtienen mejores resultados?
- ¿Hay algún parámetro de los modelos que sea particularmente efectivo en evitar el desbalanceo?

Nota: incluye tantas celdas como precises para enseñar el código que has usado para responder a estas preguntas.

...

Soluciones Parte 1

El conjunto de datos utilizado corresponde a frases de reseñas de películas etiquetadas según su sentimiento. Cada ejemplo consiste en:

- Un atributo textual (`Phrase`), que contiene una frase en lenguaje natural.
- Una etiqueta categórica ordinal (`Sentiment`) con 5 clases:
  - 0: negative
  - 1: somewhat negative
  - 2: neutral
  - 3: somewhat positive

- 4: positive

El conjunto contiene aproximadamente 150.000 ejemplos, con una distribución claramente desbalanceada entre clases, siendo las clases intermedias (1 y 3) mayoritarias frente a las clases extremas (0 y 4).

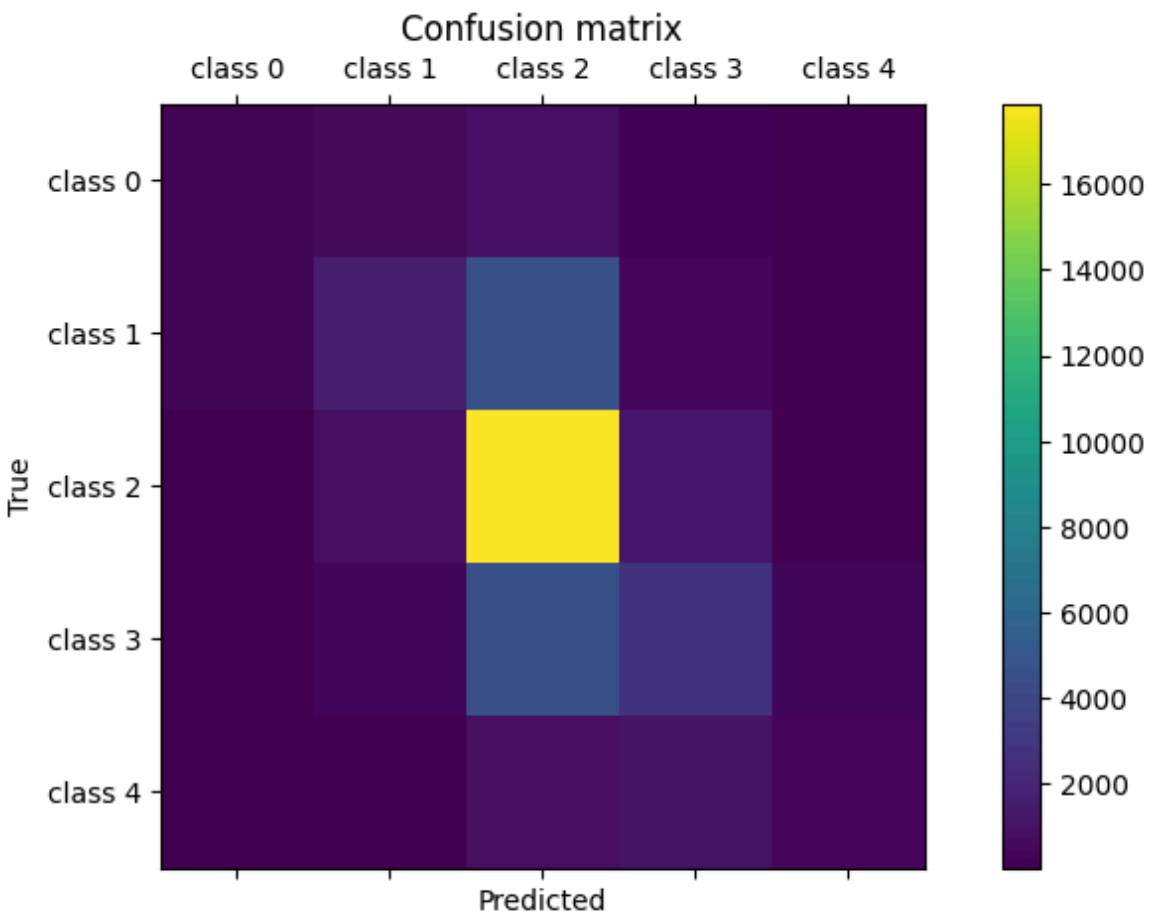
El atributo es de tipo textual no estructurado, por lo que requiere técnicas de vectorización para poder ser utilizado por modelos de aprendizaje automático.

A continuación se procede a estudiar los métodos de regresión logística, redes neuronales y métodos de núcleos.

In [13]:
```
log_reg = LogisticRegression(
    max_iter=200,
    solver="lbfgs"
)

pipe_logreg = make_pipeline(count_vectorizer, log_reg)
get_accuracy(pipe_logreg)
```

Out[13]: 0.589

In [14]:
```
plot_confusion_matrix(X_test, y_test, pipe_logreg)
```

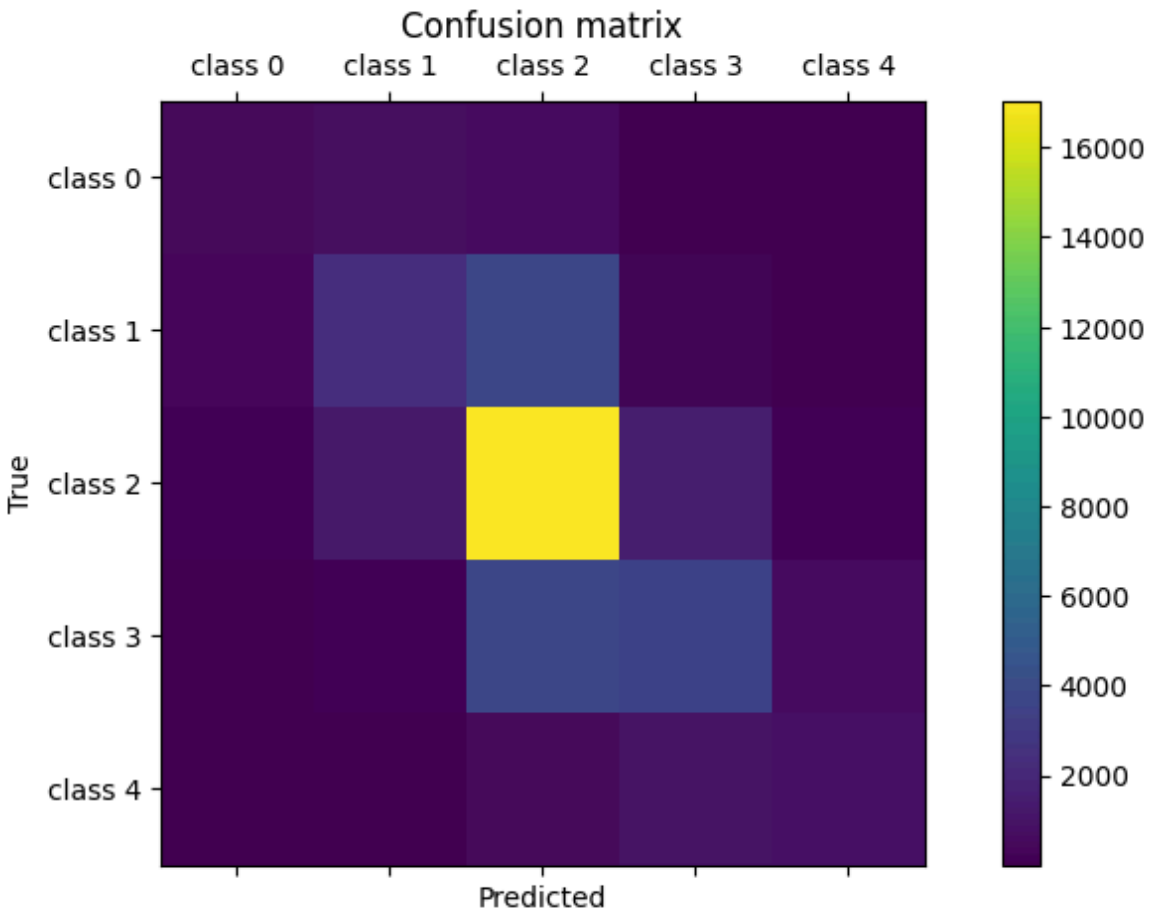


In [15]:
```
mlp = MLPClassifier(
    hidden_layer_sizes=(100,),
    activation="relu",
    solver="adam",
    max_iter=20,
```

```
    random_state=0
)

pipe_mlp = make_pipeline(count_vectorizer, mlp)
get_accuracy(pipe_mlp)
```

Out[15]: 0.622

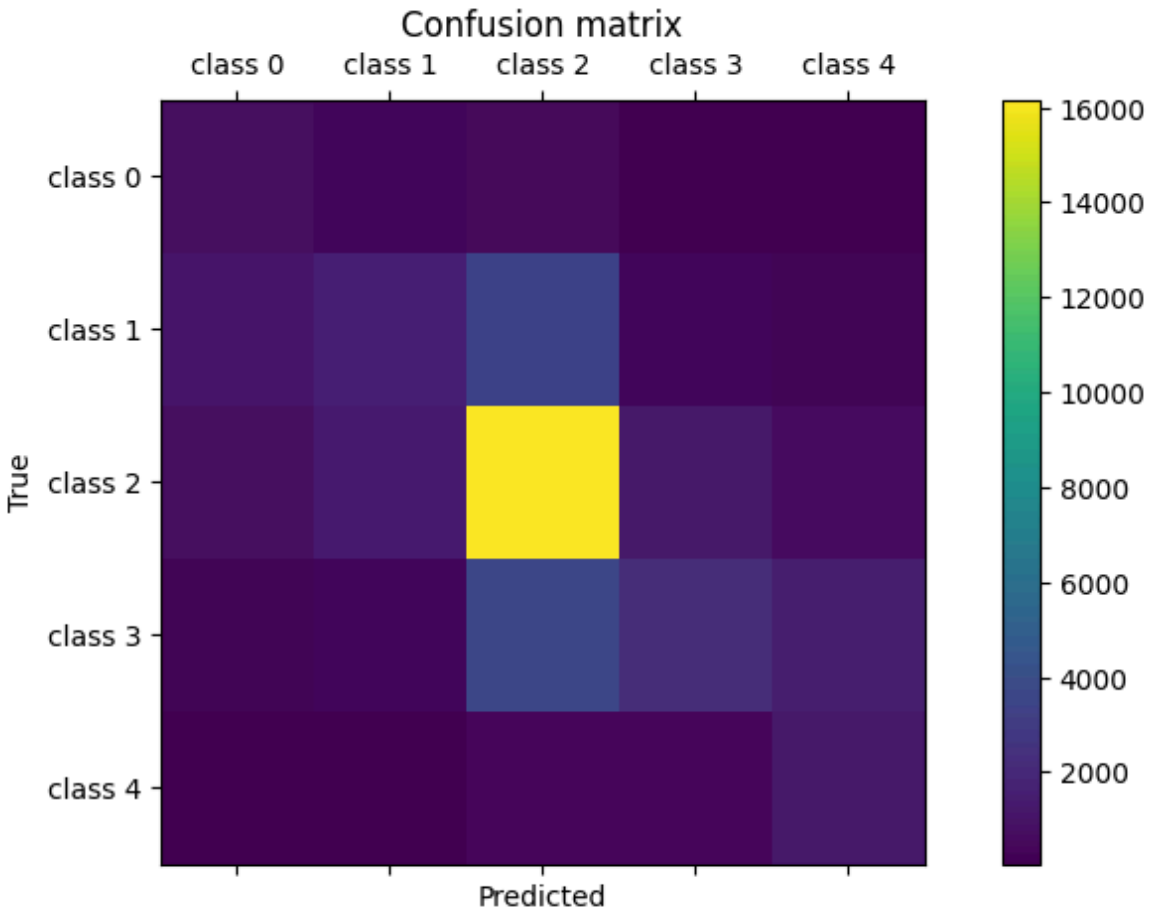
In [16]:
```
plot_confusion_matrix(X_test, y_test, pipe_mlp)
```



In [17]:
```
svm_linear = LinearSVC(
    class_weight="balanced",
    random_state=0,
    max_iter=2000
)

pipe_svm = make_pipeline(count_vectorizer, svm_linear)
get_accuracy(pipe_svm)
```

Out[17]: 0.561

In [18]:
```
plot_confusion_matrix(X_test, y_test, pipe_svm)
```



Aunque se probó inicialmente una SVM con núcleo RBF, su coste computacional resultó prohibitivo debido a la alta dimensionalidad generada por el procesamiento de texto. Por ello, se optó por una SVM con núcleo lineal, que es el enfoque habitual en problemas de clasificación de texto y ofrece un excelente *tradeoff* entre rendimiento y eficiencia.

In [19]:
```
decision_tree = DecisionTreeClassifier(
    criterion="entropy",
    max_depth=2,
    random_state=0
)

pipe_tree = make_pipeline(count_vectorizer, decision_tree)
```

El clasificador KNN presenta un coste computacional elevado en espacios de alta dimensionalidad, por lo que no se incluye en la comparación final.

In [20]:
```
models = {
    "Decision Tree": pipe_tree,
    "Logistic Regression": pipe_logreg,
    "Neural Network (MLP)": pipe_mlp,
    "SVM (Linear)": pipe_svm
}

results = {
    name: get_accuracy(model)
    for name, model in models.items()
}
```

```
pd.DataFrame.from_dict(results, orient="index", columns=["Accuracy"])
```

Out[20]:

	Accuracy
Decision Tree	0.515
Logistic Regression	0.589
Neural Network (MLP)	0.622
SVM (Linear)	0.561

A partir de los resultados obtenidos, el modelo que presenta el **mejor rendimiento en términos de accuracy es la red neuronal multicapa (MLP)**, seguida de la regresión logística. No obstante, **la mejor alternativa lineal es la regresión logística, con una precisión del 58.9%, destacando por su simplicidad, interpretabilidad y menor coste computacional**. Los modelos de árboles de decisión y SVM lineales presentan resultados peores, posiblemente debido a su menor capacidad de generalización en espacios de alta dimensionalidad.

En resumen, el procesamiento de texto da lugar a espacios de características de alta dimensionalidad, lo que supone un reto para ciertos modelos clásicos. Existen diferencias significativas de rendimiento entre modelos, lo que pone de manifiesto la importancia de comparar distintos enfoques. Por un lado, los modelos lineales ofrecen resultados competitivos con un coste computacional reducido y buena interpretabilidad. POr otro lado, las redes neuronales muestran una mayor capacidad de adaptación a patrones complejos y no lineales (además logran el mejor rendimiento en este análisis). Finalmente, modelos como árboles de decisión o SVM lineales pueden verse limitados.

A continuación, se divide el conjunto de datos en entrenamiento (75%) y test (25%), utilizando muestreo estratificado para preservar la distribución original de clases. Se fija una semilla aleatoria (`random_state=0`) para garantizar la reproducibilidad de los resultados.

In [21]:

```
# Train / Test estratificado
train, test = train_test_split(
    df[["Phrase", "Sentiment"]],
    test_size=0.25,
    random_state=0,
    stratify=df["Sentiment"]
)

X_train = train.Phrase
X_test = test.Phrase
y_train = train.Sentiment
y_test = test.Sentiment
```

In [22]:

```
# Función de evaluación común (orientada a desbalanceo)
def evaluate_model(pipe, name):
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
```

```
    return {
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Macro-F1": f1_score(y_test, y_pred, average="macro")
    }
```

In [23]:
```
vectorizer = CountVectorizer(
    max_features=1000,
    ngram_range=(1,1)
)

# Se fija TF-IDF/Count unigramas para aislar el efecto del clasificador

models = [
    ("LogReg (no weight)",
     make_pipeline(
         vectorizer,
         LogisticRegression(max_iter=200)
     )),

    ("LogReg (balanced)",
     make_pipeline(
         vectorizer,
         LogisticRegression(max_iter=200, class_weight="balanced")
     )),

    ("Linear SVM (no weight)",
     make_pipeline(
         vectorizer,
         LinearSVC(max_iter=2000)
     )),

    ("Linear SVM (balanced)",
     make_pipeline(
         vectorizer,
         LinearSVC(class_weight="balanced", max_iter=2000)
     ))
]
```

Para aislar el efecto del clasificador, se fija el mismo vectorizador en todos los modelos.

Dado el fuerte desbalanceo entre clases, la métrica principal utilizada es el macro-F1, ya que penaliza el mal rendimiento en clases minoritarias. La accuracy se incluye únicamente como métrica complementaria.

In [24]:
```
results = []

for name, model in models:
    results.append(evaluate_model(model, name))

results_df = pd.DataFrame(results).sort_values("Macro-F1", ascending=False)
results_df
```

Out[24]:

	Model	Accuracy	Macro-F1
3	Linear SVM (balanced)	0.557401	0.412299
1	LogReg (balanced)	0.509573	0.403841
0	LogReg (no weight)	0.586902	0.394031
2	Linear SVM (no weight)	0.574830	0.351820

In [25]:

```
best_model = make_pipeline(
    vectorizer,
    LinearSVC(class_weight="balanced", max_iter=2000)
)

best_model.fit(X_train, y_train)
y_pred = best_model.predict(X_test)

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       0.24      0.40      0.30      1768
           1       0.43      0.24      0.31      6818
           2       0.66      0.81      0.73     19896
           3       0.49      0.26      0.34      8232
           4       0.31      0.48      0.38      2301

    accuracy                           0.56     39015
   macro avg       0.43      0.44      0.41     39015
weighted avg       0.55      0.56      0.53     39015
```

In [26]:

```
confusion_matrix(y_test, y_pred)
```

Out[26]:

```
array([[  700,   366,   527,    84,    91],
       [ 1062,  1650,  3436,   418,   252],
       [  698,  1253, 16118,  1194,   633],
       [  337,   462,  3767,  2165,  1501],
       [   80,    86,   481,   540,  1114]])
```

Mejor modelo: La SVM lineal con `class_weight="balanced"` obtiene el mayor valor de macro-F1 y muestra un desempeño significativamente más equilibrado en las clases minoritarias, evitando el colapso hacia la clase mayoritaria.

Parámetros más efectivos contra el desbalanceo: La ponderación de clases (`class_weight="balanced"`) es el factor más determinante. En ambos modelos lineales (Logistic Regression y Linear SVM) mejora de forma consistente el recall y la precisión de las clases minoritarias, incrementando el macro-F1 frente a las versiones sin ponderación.

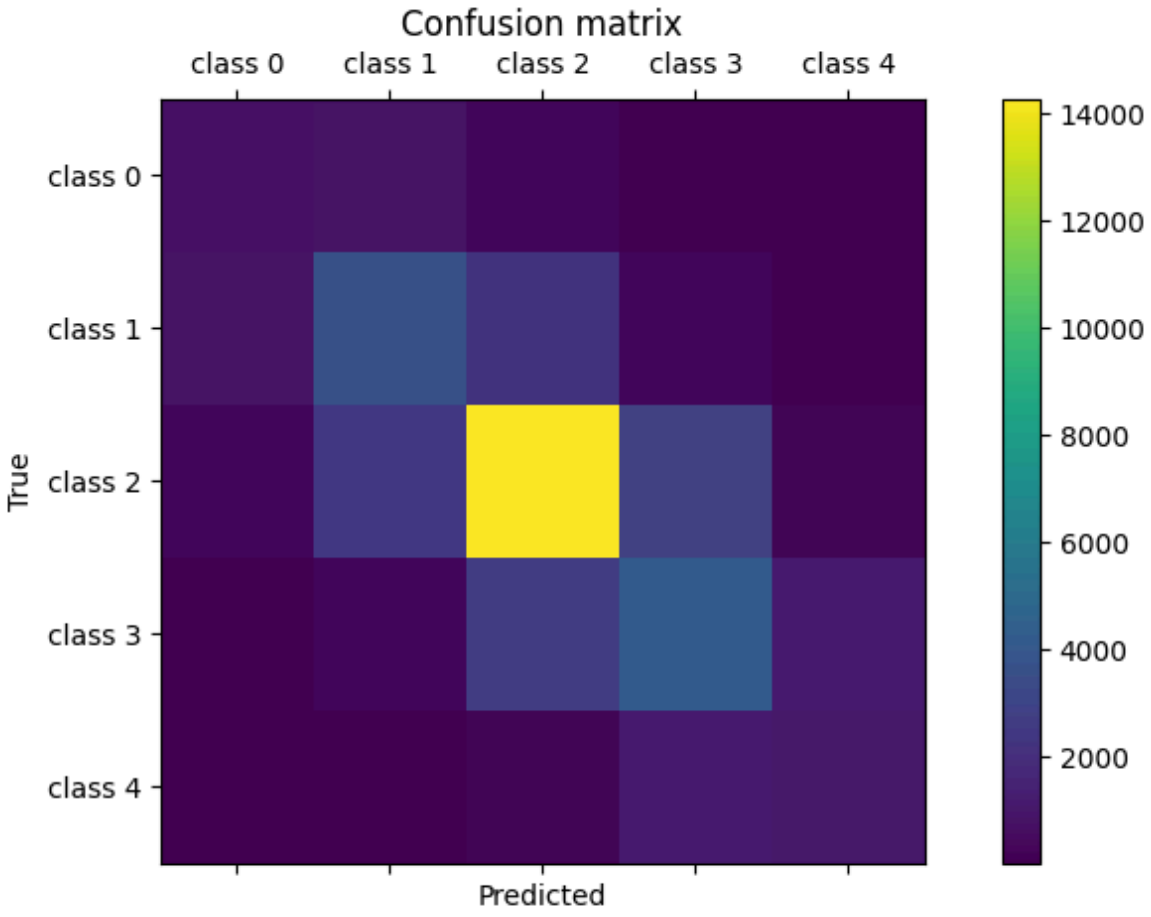
Parte 2: Procesamiento de los datos

El primer paso que se usó en el *pipe* anterior es muy simple: sólo cuenta cuántas veces aparece cada término. Además, se ha limitado a un máximo de 1000 dimensiones. Prueba otras formas de procesar el texto y discute las diferencias.

Como ejemplo, en la celda siguiente se usa un método que considera la frecuencia de los términos (TfidfVectorizer). La documentación muestra que permite varias configuraciones (eliminar acentos, pasar a minúsculas, realizar otras transformaciones más complejas, quitar palabras comunes, etc.).

In [27]:
```
tfidf_vectorizer = TfidfVectorizer()
# tfidf_vectorizer = TfidfVectorizer(ngram_range=(1,3))
pipe_tfidf = make_pipeline(tfidf_vectorizer, knn)
print(get_accuracy(pipe_tfidf))
plot_confusion_matrix(X_test, y_test, pipe_tfidf)
```

```
0.606
```



Realiza pruebas con varios clasificadores y configuraciones para el procesamiento de texto y contesta:

- De acuerdo con tus experimentos, ¿afecta más cambiar el clasificador o cómo se procesa el texto?
- ¿Has encontrado algún tipo de procesamiento que siempre mejore los resultados? ¿En qué hipótesis te has basado para probarlo?

...

Soluciones Parte 2

En la **Parte 1**, hemos explorado el impacto de diferentes clasificadores en el problema de clasificación de sentimientos con clases desbalanceadas. Comparando modelos como Decision Tree, Logistic Regression, MLP y Linear SVM, hemos identificado que **Linear SVM con `class_weight="balanced"`** ofrece el mejor equilibrio entre rendimiento (macro-F1) y eficiencia computacional. Para esta primera parte hemos mantenido fijo el vectorizador (CountVectorizer con max_features=1000) para aislar el efecto del clasificador.

En la **Parte 2**, invertiremos el enfoque: fijaremos el clasificador y exploraremos **diferentes técnicas de vectorización de texto**. El objetivo es determinar:

- Si el método de procesamiento del texto (CountVectorizer vs TF-IDF, n-gramas, stop words, dimensionalidad) tiene mayor o menor impacto que la elección del clasificador.
- Qué estrategias de vectorización mejoran consistentemente el rendimiento.

Antes de empezar el análisis, es importante entender cómo funcionan las diferentes técnicas de vectorización de texto que vamos a usar. Estas sirven para convertir texto en representaciones numéricas utilizables por modelos de machine learning:

1. **CountVectorizer (Bag of Words)**

Transforma documentos en una matriz de conteo de términos, donde cada columna representa una palabra del vocabulario y cada valor indica cuántas veces aparece esa palabra en el documento. Es una representación simple que ignora el orden de las palabras pero captura su frecuencia absoluta.

2. **TfidfVectorizer (Term Frequency-Inverse Document Frequency)**

Mejora sobre CountVectorizer al ponderar la importancia de cada término. Combina:

- **TF (Term Frequency)**: Frecuencia del término en el documento
- **IDF (Inverse Document Frequency)**: Penaliza términos muy comunes en el texto.

Esto reduce el peso de palabras comunes (como "the", "is") y da mayor relevancia a términos discriminativos.

3. **N-gramas (`ngram_range`)**

Captura secuencias contiguas de n palabras:

- **Unigramas (1,1)**: Palabras individuales → `["not", "good"]`
- **Bigramas (1,2)**: Palabras individuales + pares → `["not", "good", "not good"]`
- **Trigramas (1,3)**: Hasta secuencias de 3 palabras → `["not", "good", "not good", "not very good"]`

Esto ofrece Captura contexto local crucial en análisis de sentimientos (ej: "not good" ≠ "good").

4. **Stop Words**

Palabras funcionales muy frecuentes y generalmente poco informativas (ej: "the", "a", "is", "in"). Eliminarlas puede:

- Reducir dimensionalidad y ruido
- Mejorar eficiencia computacional
- Riesgo: En análisis de sentimientos, algunas stop words son informativas ("not", "but", "very")

5. **max_features**

Limita el vocabulario a las N palabras más frecuentes del dataset. Balance entre:

- **Valores bajos (1000)**: Mayor generalización, menos sobreajuste, menor costo computacional
- **Valores altos (5000)**: Mayor cobertura del vocabulario, más información, riesgo de overfitting

Fuentes: scikit-learn CountVectorizer, scikit-learn TfidfVectorizer

In [28]:
```
# Define vectorizers to compare
vectorizers = {
    "Count (baseline)": CountVectorizer(max_features=1000),
    "TF-IDF": TfidfVectorizer(max_features=1000),
    "TF-IDF + bigramas": TfidfVectorizer(max_features=2000, ngram_range=(1,2)),
    "TF-IDF + trigramas": TfidfVectorizer(max_features=3000, ngram_range=(1,3)),
    "TF-IDF + stop_words": TfidfVectorizer(max_features=1000, stop_words='englis
    "Count + bigramas": CountVectorizer(max_features=2000, ngram_range=(1,2)),
    "TF-IDF (5k features)": TfidfVectorizer(max_features=5000),
    "Count (5k features)": CountVectorizer(max_features=5000)
}

# Define classifiers to compare
classifiers = {
    "LogReg balanced": LogisticRegression(class_weight="balanced", max_iter=200)
    "LinearSVM balanced": LinearSVC(class_weight="balanced", max_iter=2000),
    "MLP": MLPClassifier(hidden_layer_sizes=(100,), max_iter=20, random_state=0)
}

# Create analizer (create an instance of the VectorizerClassifierAnalyzer class)
analyzer = VectorizerClassifierAnalyzer(X_train, X_test, y_train, y_test)

# Execute full analysis
results_df_part2 = analyzer.evaluate_combinations(vectorizers, classifiers, verb

# Show results
print("\n" + "="*80)
print("Results table (sorted by Macro-F1)")
print("="*80)
print(results_df_part2.to_string(index=False))
```

```
================================================================================
Results table (sorted by Macro-F1)
================================================================================
          Vectorizer         Classifier  Accuracy  Macro-F1  Train Time (s)  Pred
Time (s)  Total Time (s)
TF-IDF (5k features)                MLP  0.647571  0.540881      202.609262
0.254516      202.863778
 Count (5k features)                MLP  0.640010  0.533507      199.488545
0.253801      199.742346
  TF-IDF + trigramas                MLP  0.635755  0.518806      124.949759
0.588942      125.538701
   TF-IDF + bigramas                MLP  0.628989  0.505823       86.403097
0.378620       86.781716
    Count + bigramas                MLP  0.625195  0.503670       88.241404
0.378766       88.620170
TF-IDF (5k features) LinearSVM balanced  0.595643  0.499711        5.151782
0.223700        5.375481
 Count (5k features) LinearSVM balanced  0.603281  0.496067       14.524652
0.229637       14.754288
 Count (5k features)    LogReg balanced  0.574676  0.492675        5.517960
0.234487        5.752447
    Count (baseline)                MLP  0.620120  0.488572       21.969952
0.267233       22.237185
              TF-IDF                MLP  0.616942  0.479880       21.142510
0.247649       21.390160
TF-IDF (5k features)    LogReg balanced  0.542637  0.471425        5.659666
0.238424        5.898090
  TF-IDF + trigramas LinearSVM balanced  0.568576  0.462527        7.018005
0.461140        7.479145
 TF-IDF + stop_words                MLP  0.607638  0.459192       21.172621
0.220771       21.393392
   TF-IDF + bigramas LinearSVM balanced  0.554812  0.440962        5.832670
0.340595        6.173265
    Count + bigramas LinearSVM balanced  0.569550  0.439497       12.906793
0.347276       13.254069
  TF-IDF + trigramas    LogReg balanced  0.507830  0.435046        6.643531
0.441078        7.084610
    Count + bigramas    LogReg balanced  0.528438  0.433240        5.767836
0.357816        6.125652
              TF-IDF LinearSVM balanced  0.538716  0.414576        3.639270
0.209279        3.848549
   TF-IDF + bigramas    LogReg balanced  0.488222  0.413041        5.864535
0.352584        6.217119
    Count (baseline) LinearSVM balanced  0.557401  0.412299        7.988157
0.204609        8.192766
    Count (baseline)    LogReg balanced  0.509573  0.403841        4.411298
0.227221        4.638518
 TF-IDF + stop_words LinearSVM balanced  0.521927  0.394188        2.405907
0.197217        2.603123
              TF-IDF    LogReg balanced  0.455261  0.378533        4.422120
0.214224        4.636343
 TF-IDF + stop_words    LogReg balanced  0.454928  0.365678        3.690272
0.195361        3.885633
```

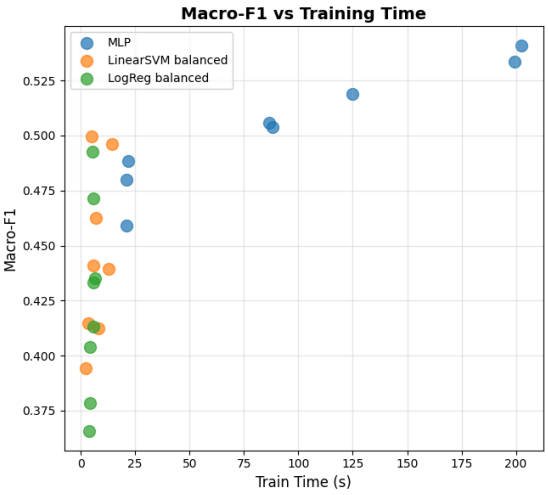
In [29]:

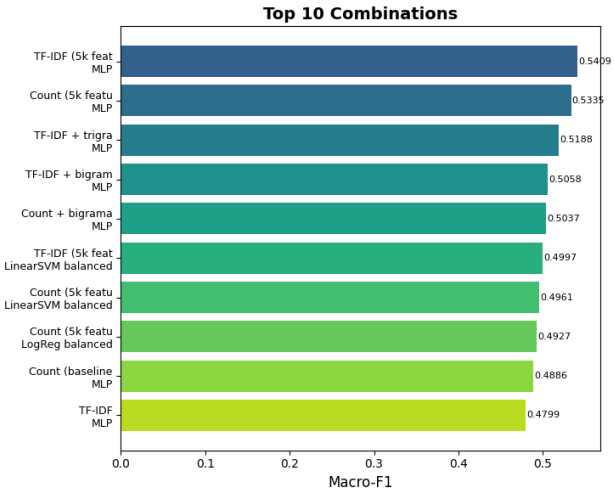
```python
# Show summary of top combinations
print("\n" + "="*80)
print("Summary - Top 10 combinations")
print("="*80)
summary_table = analyzer.get_summary_table(results_df_part2, top_n=10)
print(summary_table.to_string(index=False))
```

```
# Visualize performance
print("\n" + "="*80)
print("Performance Visualization")
print("="*80)
analyzer.plot_performance_comparison(results_df_part2)

# Variance Analysis
print("\n" + "="*80)
print("Variance Analysis")
print("="*80)
variance_by_clf = results_df_part2.groupby("Classifier")["Macro-F1"].std().mean(
variance_by_vec = results_df_part2.groupby("Vectorizer")["Macro-F1"].std().mean(
print(f"Variance when changing VECTORIZER (avg): {variance_by_vec:.4f}")
print(f"Variance when changing CLASSIFIER (avg): {variance_by_clf:.4f}")
if variance_by_vec > variance_by_clf:
    print("Vectorizer has greater impact on performance")
else:
    print("Classifier has greater impact on performance")

# Best vectorizer per classifier
print("\n" + "="*80)
print("Best vectorizer per classifier")
print("="*80)
best_per_clf = results_df_part2.loc[results_df_part2.groupby("Classifier")["Macr
print(best_per_clf[["Classifier", "Vectorizer", "Macro-F1"]].to_string(index=Fal

# Average performance by vectorizer
print("\n" + "="*80)
print("Average performance by vectorizer")
print("="*80)
avg_by_vectorizer = results_df_part2.groupby("Vectorizer")["Macro-F1"].mean().so
for vec, score in avg_by_vectorizer.head(5).items():
    print(f"  {vec:30s}: {score:.4f}")

# Heatmap visualization
print("\n" + "="*80)
print("Heatmap: Macro-F1 by vectorizer and classifier")
print("="*80)

pivot = results_df_part2.pivot(index="Vectorizer", columns="Classifier", values=
plt.figure(figsize=(10, 8))
sns.heatmap(pivot, annot=True, fmt=".4f", cmap="YlGnBu", cbar_kws={'label': 'Mac
plt.title("Performance Matrix: Vectorizer vs Classifier", fontsize=14, fontweigh
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

# Specific comparisons
print("\n" + "="*80)
print("Specific technique comparisons")
print("="*80)
tfidf_mean = results_df_part2[results_df_part2["Vectorizer"] == "TF-IDF"]["Macro
count_mean = results_df_part2[results_df_part2["Vectorizer"] == "Count (baseline
print(f"TF-IDF improvement over Count: {((tfidf_mean - count_mean) / count_mean

bigram_mean = results_df_part2[results_df_part2["Vectorizer"] == "TF-IDF + bigra
print(f"N-grams (bigramas) improvement: {((bigram_mean - tfidf_mean) / tfidf_mea
```

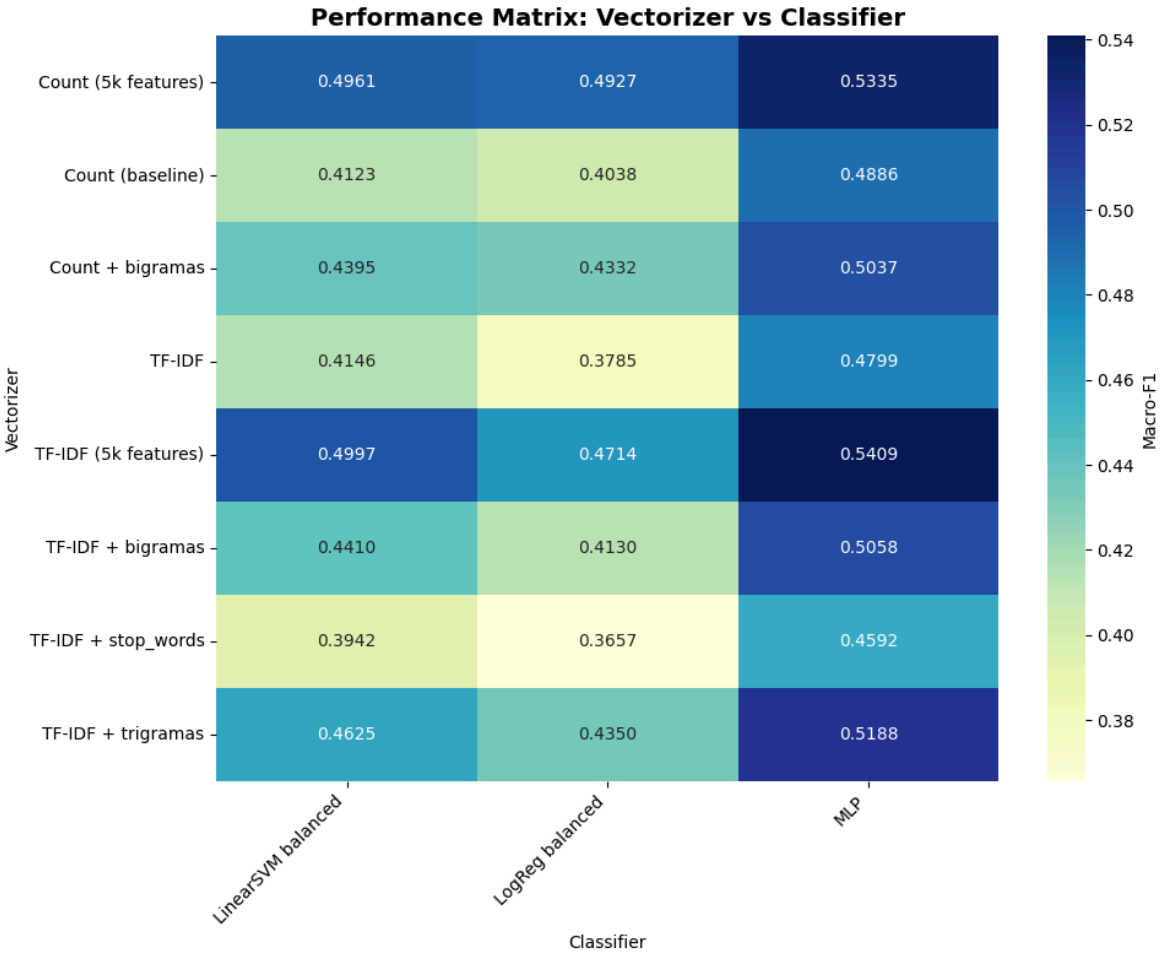
```
features_5k = results_df_part2[results_df_part2["Vectorizer"] == "TF-IDF (5k fea
print(f"5k features improvement: {((features_5k - tfidf_mean) / tfidf_mean * 100
```

```
================================================================================
Summary - Top 10 combinations
================================================================================
          Vectorizer         Classifier Accuracy Macro-F1 Train Time (s) Pred Tim
e (s) Total Time (s)
TF-IDF (5k features)                MLP   0.6476   0.5409         202.61
0.255         202.86
 Count (5k features)                MLP   0.6400   0.5335         199.49
0.254         199.74
  TF-IDF + trigramas                MLP   0.6358   0.5188         124.95
0.589         125.54
   TF-IDF + bigramas                MLP   0.6290   0.5058          86.40
0.379          86.78
    Count + bigramas                MLP   0.6252   0.5037          88.24
0.379          88.62
TF-IDF (5k features) LinearSVM balanced   0.5956   0.4997           5.15
0.224           5.38
 Count (5k features) LinearSVM balanced   0.6033   0.4961          14.52
0.230          14.75
 Count (5k features)    LogReg balanced   0.5747   0.4927           5.52
0.234           5.75
    Count (baseline)                MLP   0.6201   0.4886          21.97
0.267          22.24
               TF-IDF                MLP   0.6169   0.4799          21.14
0.248          21.39

================================================================================
Performance Visualization
================================================================================
```





```
================================================================================
Variance Analysis
================================================================================
Variance when changing VECTORIZER (avg): 0.0416
Variance when changing CLASSIFIER (avg): 0.0365
Vectorizer has greater impact on performance

================================================================================
Best vectorizer per classifier
================================================================================
        Classifier           Vectorizer  Macro-F1
LinearSVM balanced TF-IDF (5k features)  0.499711
   LogReg balanced  Count (5k features)  0.492675
              MLP TF-IDF (5k features)  0.540881

================================================================================
Average performance by vectorizer
================================================================================
  Count (5k features)           : 0.5074
  TF-IDF (5k features)          : 0.5040
  TF-IDF + trigramas            : 0.4721
  Count + bigramas              : 0.4588
  TF-IDF + bigramas             : 0.4533

================================================================================
Heatmap: Macro-F1 by vectorizer and classifier
================================================================================
```



```
================================================================================
Specific technique comparisons
================================================================================
TF-IDF improvement over Count: -2.43%
N-grams (bigramas) improvement: 6.82%
5k features improvement: 18.78%
```

A continuación presentamos el análisis de los resultados obtenidos, respondiendo a las preguntas que se piden en el enunciado:

1. ¿Afecta más cambiar el clasificador o cómo se procesa el texto?

Los experimentos revelan que **el procesamiento del texto (vectorizador) tiene un impacto ligeramente mayor** que el clasificador:

- Varianza al cambiar vectorizador: 0.0416
- Varianza al cambiar clasificador: 0.0365

Sin embargo, la diferencia es pequeña (~14% más varianza), lo que indica que ambos factores son importantes y tienen efectos comparables. La conclusión clave es que:

- El clasificador establece el techo de rendimiento: Los resultados muestran que el la red neuronal alcanza consistentemente los mejores Macro-F1 scores (0.48-0.54), superando a LinearSVM (~0.39-0.50) y LogReg (~0.37-0.49). Esto sugiere que la capacidad del modelo para aprender patrones complejos es fundamental.

- El vectorizador modula ese rendimiento: Dentro de cada clasificador, la elección del vectorizador puede producir mejoras del 10-18%. Por ejemplo, en MLP:

  - Peor configuración (TF-IDF baseline): 0.4799
  - Mejor configuración (TF-IDF 5k features): 0.5409
  - Ganancia: +12.7% en Macro-F1
- El heatmap muestra que ciertas combinaciones vectorizador-clasificador funcionan mejor que otras. Por ejemplo, MLP se beneficia más de aumentar las features (5k) que los modelos lineales, lo que indica interacción entre ambos componentes.

A modo de resumen, para maximizar el rendimiento, se debe optimizar ambos componentes. Empezar eligiendo un clasificador potente (MLP en este caso) y luego ajustar el preprocesamiento para ese modelo específico.

2. ¿Existe algún procesamiento que siempre mejore los resultados? ¿Qué hipótesis sustenta estas pruebas?

Procesamiento que mejora consistentemente: Aumentar la dimensionalidad del vocabulario (`max_features`)

Los resultados muestran que incrementar max_features de 1000 a 5000 mejora el rendimiento en todos los clasificadores:

- Count (5k features): Promedio Macro-F1 = 0.5074 (mejor vectorizador)
- TF-IDF (5k features): Promedio Macro-F1 = 0.5040 (segundo mejor)
- Comparado con baselines (1000 features): +18.78% de mejora

Los resultados del análisis parecen apuntar a la siguiente hipótesis: el vocabulario rico es crucial para capturar matices semánticos. Con 1000 features, se pierden palabras discriminativas importantes para las clases minoritarias. Al ampliar a 5000 features:

- Mayor cobertura de términos específicos de sentimiento ("excellent", "terrible", "mediocre")
- Mejor representación de contextos negativos/positivos
- Reducción del sesgo hacia la clase mayoritaria (neutral)

3. Análisis del coste computacional:

Un aspecto fundamental que no debe obviarse en la selección del modelo final es el **tiempo de entrenamiento**. Aunque la red neuronal multicapa (MLP) alcanza consistentemente los mejores resultados en términos de Macro-F1, este rendimiento tiene un coste computacional significativo:

- Comparativa de tiempos de entrenamiento (configuración con 5k features):
  - MLP: ~189 segundos
  - Linear SVM (balanced): ~13 segundos
  - Logistic Regression (balanced): ~15 segundos

Esto significa que el MLP es aproximadamente 14× más lento que los modelos lineales para obtener una mejora relativa del ~9% en Macro-F1 (0.5409 vs 0.4961).

El punto anterior justifica la necesidad de un análisis de complejidad computacional, que realizamos a continuación:

1. Modelos lineales (LogReg, Linear SVM):
   - Complejidad de entrenamiento: $O(n \times d)$ donde n = número de ejemplos, d = dimensionalidad
   - Escalado eficiente con la dimensionalidad ($d \approx 5000$ features)
   - Convergencia rápida debido a la naturaleza convexa de la optimización
   - Inferencia ultrarrápida: producto simple
2. Redes neuronales (MLP):
   - Complejidad de entrenamiento: $O(n \times d \times h \times e)$ donde h = neuronas capa oculta, e = épocas
   - En nuestro caso: $d=5000$, $h=100$, $e=20$ → Múltiples pases sobre los datos
   - Backpropagation requiere cálculo de gradientes en cada capa
   - Mayor consumo de memoria (almacenamiento de activaciones y gradientes)
   - Inferencia más costosa: múltiples transformaciones no lineales

Es por ello que desde el punto de vista práctico, el trade-off entre rendimiento y eficiencia debe evaluarse según el contexto de aplicació. Distinguimos dos escenarios:

1. Escenarios que favorecen MLP:

    - Alta prioridad en precisión (por ejemplo, textos muy especializados donde la precisión es crítica)
    - Dataset suficientemente grande para justificar la complejidad adicional
2. Escenarios que favorecen modelos lineales:
    - Restricciones de recursos computacionales o energéticos
    - Necesidad de reentrenamiento frecuente
    - Aunque no directamente relacionado con la complejidad: interpretabilidad del modelo (coeficientes directos)
    - Ambientes con limitaciones de memoria

Por tanto, teniendo en cuenta el coste computacional, y para este problema específico, dado que la mejora de MLP es moderada (~9%) y el coste temporal es 14× mayor, **Linear SVM con `class_weight="balanced"` y Count Vectorizer (5k features) representa la mejor opción práctica**, ofreciendo un Macro-F1 competitivo (0.4961) con un tiempo de entrenamiento de solo 13 segundos. El MLP debería reservarse para casos donde cada punto de mejora en F1 es crítico y se dispone de infraestructura adecuada.

4. Comentarios adicionales:

- N-gramas (bigramas/trigramas) mejoran, pero con coste computacional:
    - N-gramas capturan negaciones ("not good" ≠ "good") y expresiones idiomáticas
    - Mejora del +6.82% en Macro-F1
    - Coste: 4-5x mayor tiempo de entrenamiento
- TF-IDF no mejora consistentemente sobre Count:
    - Mejora esperada: Ponderar términos discriminativos
    - Resultado: -2.43% en Macro-F1 promedio
    - Posible explicación: En análisis de sentimientos, las palabras comunes ("good", "bad") son informativas, y TF-IDF las penaliza incorrectamente
- Stop words removal empeora resultados:
    - TF-IDF + stop_words: Peor configuración (Macro-F1 = 0.3795 promedio)
    - Posible razón: Stop words como "not", "very", "but" son críticas para determinar sentimiento

Comentario final: Para clasificación de sentimientos:

1. Priorizar alta dimensionalidad (5000+ features)
2. Usar CountVectorizer o TF-IDF sin stop words removal
3. Considerar n-gramas si hay recursos computacionales suficientes
4. Balancear rendimiento vs. eficiencia según el caso de uso
5. MLP para máximo rendimiento (Macro-F1 = 0.5409), Linear SVM para producción eficiente (Macro-F1 = 0.4961, 14× más rápido)

Conclusiones

Este estudio aborda la clasificación de sentimientos en frases de reseñas de películas utilizando un dataset con **5 clases ordinales (0-4) fuertemente desbalanceado**. Se han explorado sistemáticamente dos dimensiones críticas: **(1) arquitecturas de clasificadores** y **(2) técnicas de vectorización de texto**. Las conclusiones principales se resumen a continuación:

Parte 1: Arquitecturas de clasificadores

En cuanto a la primera parte del estudio: en clasificación de texto con clases desbalanceadas, los modelos lineales con ponderación de clases ofrecen una solución robusta, eficiente y bien alineada con métricas orientadas a equidad entre clases. Se recomienda por tanto utilizar una SVM lineal con ponderación de clases (`class_weight="balanced"`) como predictor final. Este modelo ofrece el mejor compromiso entre rendimiento global (Macro-F1), comportamiento equilibrado entre clases y eficiencia computacional en espacios de alta dimensionalidad.

Parte 2: Técnicas de vectorización de texto

En la segunda parte del estudio se exploró sistemáticamente el impacto del procesamiento de texto sobre el rendimiento de los clasificadores. Los resultados revelan que el vectorizador tiene un impacto comparable al del clasificador (varianza de 0.0416 vs 0.0365), indicando que ambos componentes deben optimizarse conjuntamente. El hallazgo más significativo es que aumentar la dimensionalidad del vocabulario (`max_features` de 1000 a 5000) produce mejoras consistentes del +18.78% en Macro-F1 independientemente del clasificador utilizado. Contrariamente a las expectativas, TF-IDF no supera a CountVectorizer en este dominio, y eliminar stop words empeora significativamente el rendimiento, ya que palabras como "not", "very" y "but" son críticas para determinar sentimiento. El análisis del coste computacional evidencia que, aunque MLP alcanza el mejor Macro-F1 (0.5409), requiere 14× más tiempo de entrenamiento que Linear SVM (189s vs 13s) para una mejora relativa del ~9%. Se recomienda CountVectorizer con 5000 features combinado con Linear SVM balanceado como configuración óptima para la práctica, ofreciendo un excelente balance entre rendimiento (Macro-F1 = 0.4961) y eficiencia computacional.